4.7 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-4.1 Describe a recursive algorithm for finding the maximum element in a sequence, S, of n elements. What is your running time and space usage?

R-4.2 Draw the recursion trace for the computation of $power(2,5)$, using the traditional function implemented in Code Fragment 4.11.

R-4.3 Draw the recursion trace for the computation of $power(2,18)$, using the repeated squaring algorithm, as implemented in Code Fragment 4.12.

R-4.4 Draw the recursion trace for the execution of function reverse(S, 0, 5) (Code Fragment 4.10) on S = [4, 3, 6, 2, 6].

R-4.5 Draw the recursion trace for the execution of function PuzzleSolve$(3,S,U)$ (Code Fragment 4.14), where S is empty and $U = \{a,b,c,d\}$.

R-4.6 Describe a recursive function for computing the nth ***Harmonic number***, $H_n = \sum_{i=1}^{n} 1/i$.

R-4.7 Describe a recursive function for converting a string of digits into the integer it represents. For example, `'13531'` represents the integer 13,531.

R-4.8 Isabel has an interesting way of summing up the values in a sequence A of n integers, where n is a power of two. She creates a new sequence B of half the size of A and sets $B[i] = A[2i] + A[2i+1]$, for $i = 0,1,\ldots,(n/2)-1$. If B has size 1, then she outputs $B[0]$. Otherwise, she replaces A with B, and repeats the process. What is the running time of her algorithm?

Creativity

C-4.9 Write a short recursive Python function that finds the minimum and maximum values in a sequence without using any loops.

C-4.10 Describe a recursive algorithm to compute the integer part of the base-two logarithm of n using only addition and integer division.

C-4.11 Describe an efficient recursive function for solving the element uniqueness problem, which runs in time that is at most $O(n^2)$ in the worst case without using sorting.

C-4.12 Give a recursive algorithm to compute the product of two positive integers, m and n, using only addition and subtraction.

C-4.13 In Section 4.2 we prove by induction that the number of *lines* printed by a call to draw_interval(c) is $2^c - 1$. Another interesting question is how many *dashes* are printed during that process. Prove by induction that the number of dashes printed by draw_interval(c) is $2^{c+1} - c - 2$.

C-4.14 In the ***Towers of Hanoi*** puzzle, we are given a platform with three pegs, a, b, and c, sticking out of it. On peg a is a stack of n disks, each larger than the next, so that the smallest is on the top and the largest is on the bottom. The puzzle is to move all the disks from peg a to peg c, moving one disk at a time, so that we never place a larger disk on top of a smaller one. See Figure 4.15 for an example of the case $n = 4$. Describe a recursive algorithm for solving the Towers of Hanoi puzzle for arbitrary n. (Hint: Consider first the subproblem of moving all but the nth disk from peg a to another peg using the third as "temporary storage.")

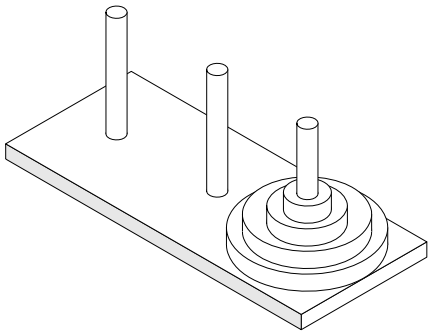


Figure 4.15: An illustration of the Towers of Hanoi puzzle.

C-4.15 Write a recursive function that will output all the subsets of a set of n elements (without repeating any subsets).

C-4.16 Write a short recursive Python function that takes a character string s and outputs its reverse. For example, the reverse of `'pots&pans'` would be `'snap&stop'`.

C-4.17 Write a short recursive Python function that determines if a string s is a palindrome, that is, it is equal to its reverse. For example, `'racecar'` and `'gohangasalamiimalasagnahog'` are palindromes.

C-4.18 Use recursion to write a Python function for determining if a string s has more vowels than consonants.

C-4.19 Write a short recursive Python function that rearranges a sequence of integer values so that all the even values appear before all the odd values.

C-4.20 Given an unsorted sequence, S, of integers and an integer k, describe a recursive algorithm for rearranging the elements in S so that all elements less than or equal to k come before any elements larger than k. What is the running time of your algorithm on a sequence of n values?

C-4.21 Suppose you are given an n-element sequence, S, containing distinct integers that are listed in increasing order. Given a number k, describe a recursive algorithm to find two integers in S that sum to k, if such a pair exists. What is the running time of your algorithm?

C-4.22 Develop a nonrecursive implementation of the version of power from Code Fragment 4.12 that uses repeated squaring.

Projects

P-4.23 Implement a recursive function with signature find(path, filename) that reports all entries of the file system rooted at the given path having the given file name.

P-4.24 Write a program for solving summation puzzles by enumerating and testing all possible configurations. Using your program, solve the three puzzles given in Section 4.4.3.

P-4.25 Provide a nonrecursive implementation of the draw_interval function for the English ruler project of Section 4.1.2. There should be precisely $2^c - 1$ lines of output if c represents the length of the center tick. If incrementing a counter from 0 to $2^c - 2$, the number of dashes for each tick line should be exactly one more than the number of consecutive 1's at the end of the binary representation of the counter.

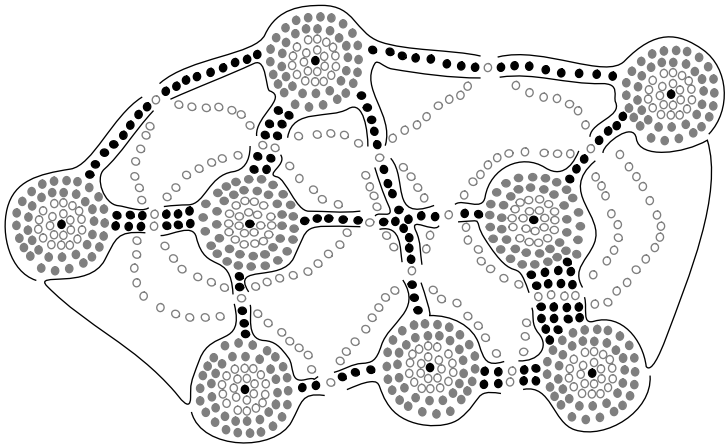
P-4.26 Write a program that can solve instances of the Tower of Hanoi problem (from Exercise C-4.14).

P-4.27 Python's os module provides a function with signature walk(path) that is a generator yielding the tuple (dirpath, dirnames, filenames) for each subdirectory of the directory identified by string path, such that string dirpath is the full path to the subdirectory, dirnames is a list of the names of the subdirectories within dirpath, and filenames is a list of the names of non-directory entries of dirpath. For example, when visiting the cs016 subdirectory of the file system shown in Figure 4.6, the walk would yield ('/user/rt/courses/cs016', ['homeworks', 'programs'], ['grades']). Give your own implementation of such a walk function.

Chapter Notes

The use of recursion in programs belongs to the folkore of computer science (for example, see the article of Dijkstra [36]). It is also at the heart of functional programming languages (for example, see the classic book by Abelson, Sussman, and Sussman [1]). Interestingly, binary search was first published in 1946, but was not published in a fully correct form until 1962. For further discussions on lessons learned, please see papers by Bentley [14] and Lesuisse [68].

Chapter 5 Array-Based Sequences



Contents

5.1 Python's Sequence Types 184
5.2 Low-Level Arrays 185
5.2.1 Referential Arrays 187
5.2.2 Compact Arrays in Python 190
5.3 Dynamic Arrays and Amortization 192
5.3.1 Implementing a Dynamic Array 195
5.3.2 Amortized Analysis of Dynamic Arrays 197
5.3.3 Python's List Class 201
5.4 Efficiency of Python's Sequence Types 202
5.4.1 Python's List and Tuple Classes 202
5.4.2 Python's String Class 208
5.5 Using Array-Based Sequences 210
5.5.1 Storing High Scores for a Game 210
5.5.2 Sorting a Sequence 214
5.5.3 Simple Cryptography 216
5.6 Multidimensional Data Sets 219
5.7 Exercises 224